



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3843 – SPECIAL TOPIC IN DATA ENGINEERING

SECTION 2

TUTORIAL 4:

CLAUDE AND PIPELINE

LECTURER: DR. ARYATI BINTI BAKRI

GROUP 2

STUDENT NAME	MATRIC NO
CHAU YING JIA	A23CS0213
LAU YEE WEN	A23CS0099
POH LOK YEE	A23CS0262

Table of Contents

1.0 AI-assisted Data Engineering.....	3
2.0 Claude/Ai agent Used in the Tutorial.....	4
2.1 AI Model - Dataset Profiling Stage.....	4
2.2 AI Model 1 - Issue Detection Stage.....	4
3.0 Steps to Do Prompt and Coding.....	5
Step 1: Create the Dataset.....	5
Step 2: Designing the Airia AI Agent Workflow.....	6
Step 3: Configuring the Prompt for Dataset Profiling.....	7
Step 4: Configuring the Prompt for Data Quality Issue Detection.....	8
Step 5: Configuring the Prompt for Report Generation.....	9
Step 6: Building the Airflow DAG.....	10
Step 7: Extracting and Loading the Dataset.....	11
Step 8: Calling the Airia AI Agent API.....	12
Step 9: Receiving the AI-Generated Data Quality Report.....	14
Step 10: Cleaning the Dataset Using pandas.....	14
Step 11: Saving the Cleaned Output.....	15
Step 12: Executing the Pipeline in Airflow.....	16
4.0 Result and Discussion.....	18
4.1 Use Case Diagram.....	18
4.2 Pipeline Execution Flow.....	19
4.3 Result.....	20
5.0 Reflection.....	22
6.0 References.....	25

1.0 AI-assisted Data Engineering

Artificial Intelligence assisted data engineering refers to the application of AI technologies to automate, enhance and optimize various stages of the data engineering lifecycle. Traditional data engineering workflows often rely heavily on manually written rules, scripts and validation procedures to process and manage data. However, these approaches can become difficult to maintain as data volume, complexity and variety increase (Peddisetti, 2023).

AI-assisted data engineering provides intelligent systems that can analyze datasets, identify patterns, detect anomalies, generate code and assist with decision-making processes. These systems help data engineers reduce repetitive manual tasks and improve the efficiency of data processing pipelines.

One of the most important applications of AI-assisted data engineering is data quality management. Data quality issues such as missing values, duplicate records, invalid formats and inconsistent data can negatively affect analytics and business decisions. Traditionally, these issues are detected through predefined validation rules. However, AI models can perform contextual analysis and identify data quality problems without requiring extensive manual rule creation.

In this tutorial, an AI agent was integrated into a data pipeline to perform automated data quality assessment. The AI agent analyzed a customer dataset, identified potential issues, classified their severity and generated recommendations for improvement. This demonstrates how AI can support data engineers in maintaining high-quality datasets while reducing manual effort.

Furthermore, AI-assisted data engineering aligns with modern industry trends where organizations increasingly incorporate AI capabilities into ETL (Extract, Transform, Load) and ELT (Extract, Load, Transform) pipelines. AI can be used not only for data quality assessment but also for schema detection, metadata generation, pipeline monitoring, anomaly detection and automated documentation (Tandon, 2025).

2.0 Claude/Ai agent Used in the Tutorial

This tutorial utilized Claude Haiku 4.5 which is a Large Language Model (LLM) used within the Airia AI Agent platform. Claude Haiku 4.5 served as the core intelligence component responsible for analyzing the dataset and generating data quality insights. An AI agent is a software system that uses artificial intelligence to perform tasks autonomously based on user instructions. Unlike traditional programs that follow fixed rules, AI agents can learn from information, understand context and generate dynamic responses. In this tutorial, the Airia AI Agent was designed as a multi-stage workflow consisting of three AI processing stages shown in Figure 2.1 below.

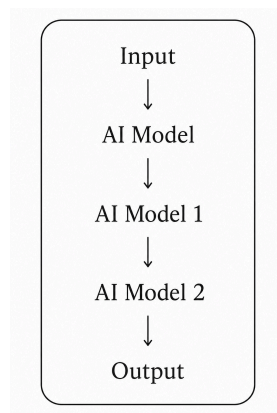


Figure 2.1 Workflow in Airia AI Agent

2.1 AI Model - Dataset Profiling Stage

The first AI model was used to perform dataset profiling. In this stage, the dataset was examined to understand its overall structure and condition before any deeper checking was carried out. The model identified important dataset information such as the column names, data types, number of records, missing values and basic data patterns. For example, it could check whether the age column contained numerical values, whether the email column followed a text format and whether any columns had empty values. This stage was important because it provided an initial overview of the dataset. It also helped the next AI model understand what kind of data quality issues should be detected. In simple words, this stage worked like an early inspection process, where the dataset was reviewed before further analysis was performed.

2.2 AI Model 1 - Issue Detection Stage

The second AI model was used to detect data quality problems in the dataset. Based on the dataset profile generated by the first AI model, this stage focused on identifying errors or unusual values that could affect the accuracy of the data. The model checked for missing values, duplicate records, invalid email formats, invalid dates, negative values and unrealistic values. For example, an age value such as 150 could be considered unrealistic, while a spending value below 0 could be considered invalid. This stage was important because poor-quality data can lead to incorrect analysis and unreliable reporting. By

detecting these issues automatically, the AI model helped reduce the amount of manual checking required by data engineers.

2.3 AI Model 2 - Report Generation Stage

The third AI model was used to generate the final data quality report. After the issues had been identified, the model summarized the results in a clear and readable format. The report included the total number of records, the types of issues detected, examples of problematic records and recommended solutions. This made the results easier to understand for users who may not be familiar with the raw dataset. Instead of manually reading through many rows of data, the user could refer to the report to quickly understand what problems existed and how they could be fixed. This stage showed how AI could support the data engineering workflow by making data quality analysis faster, clearer and more practical.

3.0 Steps to Do Prompt and Coding

This section explains the implementation steps for the AI-assisted data quality pipeline. The implementation includes two main parts: prompt configuration in Airia AI Agent and coding implementation using Apache Airflow, Python, pandas, and the Airia API.

Step 1: Create the Dataset

A dummy customer dataset named `customer_data.csv` was created for this tutorial. The dataset contains customer information such as customer ID, name, email, age, country, signup date, and spending amount. Several data quality issues were intentionally added into the dataset so that the AI agent could detect and report them.

As shown in Figure 3.1, the dataset contains common data quality problems such as missing values, invalid email formats, duplicate customer records, unrealistic age values, invalid dates, and negative spending amounts. These issues make the dataset suitable for testing the AI-assisted data quality pipeline.

```

customer_data.csv
1  customer_id,name,email,age,signup_date,country,total_spend
2  1,Aisha Rahman,aisha.rahman@example.com,22,2025-01-15,Malaysia,120.50
3  2,Ben Tan,ben.tan@example.com,34,2025-02-03,Singapore,250.00
4  3,Chloe Lim,chloe.lim@example.com,,2025-02-21,Malaysia,89.90
5  4,David Wong,david.wong@example.com,-5,2025-03-01,Malaysia,45.00
6  5,Eva Lee,eva.lee@example.com,29,not-a-date,Thailand,310.75
7  6,Farid Khan,farid.khan@example.com,41,2025-03-18,,0.00
8  7,Aisha Rahman,aisha.rahman@example.com,22,2025-01-15,Malaysia,120.50
9  8,Gina Ong,gina.ong@example.com,150,2025-04-09,Indonesia,-20.00
10 9,Hazim Noor,hazim.noor@example.com,27,2025-04-12,Malaysia,77.20
11 10,Irene Koh,irene.koh@example.com,31,2025-04-15,Singapore,188.00
12 11,Jason Yap,jason.yap@example.com,45,2025-04-18,Malaysia,402.35
13 12,Kavitha Nair,kavitha.nair@example.com,38,2025-04-20,India,215.80
14 13,Lucas Chen,lucas.chen@example.com,24,2025-04-25,Malaysia,95.60
15 14,Maya Abdullah,maya.abdullah@example.com,33,2025-04-27,Malaysia,166.40
16 15,Nora Ismail,nora.ismail@example.com,28,2025-05-01,Malaysia,140.00
17 16,Omar Aziz,omar.aziz@example.com,52,2025-05-04,Brunei,520.10
18 17,Prisha Kumar,prisha.kumar@example.com,36,2025-05-06,India,275.90
19 18,Qistina Zulkifli,qistina.zulkifli@example.com,19,2025-05-09,Malaysia,33.40
20 19,Raj Singh,raj.singh@example.com,44,2025-05-12,India,390.00
21 20,Siti Aminah,siti.aminah@example.com,26,2025-05-14,Malaysia,180.75
22 21,Thomas Lee,thomas.lee@example.com,30,2025-05-17,Singapore,225.30
23 22,Usha Devi,usha.devi@example.com,47,2025-05-19,India,315.20
24 23,Vivian Ho,vivian.ho@example.com,35,2025-05-22,Malaysia,260.00
25 24,Wei Ming,wei.ming@example.com,23,2025-05-25,Malaysia,72.90
26 25,Xavier Lim,xavier.lin@example.com,40,2025-05-27,Singapore,430.45

```

Figure 3.1: Sample customer_data.csv dataset.

The purpose of preparing this dataset was to simulate a real-world situation where raw data may contain errors before it is used for analysis or reporting.

Step 2: Designing the Airia AI Agent Workflow

After preparing the dataset, an AI agent was created in Airia. The Airia workflow was designed using multiple AI model blocks connected in sequence. As shown in Figure 3.2, the workflow follows this structure:

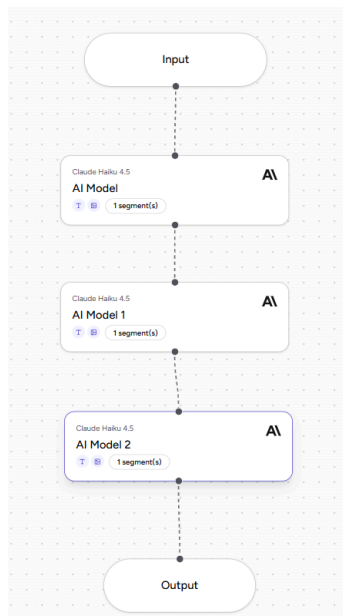


Figure 3.2: Airia AI Agent workflow

Each AI model block has a different responsibility. The first AI model performs dataset profiling, the second detects data quality issues, and the third generates the final report. This design follows the tutorial concept where AI agents are used to replace or support traditional rule-based data quality checking.

The multi-step structure is useful because the task is divided into smaller stages. Instead of asking one AI model to do everything at once, each model focuses on one specific part of the data quality process. This makes the workflow easier to understand and explain.

Step 3: Configuring the Prompt for Dataset Profiling

The first AI model was configured to analyze the dataset structure. As shown in Figure 3.3, the prompt asks the AI model to identify the data type of each column, count missing values, and detect visible patterns in the dataset.

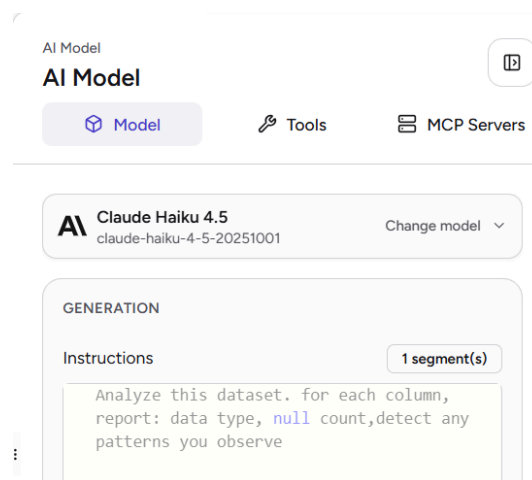


Figure 3.3: Prompt used for dataset profiling

The prompt used for this stage was:

“Analyze this dataset. For each column, report the data type, null count, and detect any patterns you observe.”

This prompt allows the AI model to perform a basic profiling process. Dataset profiling is an important step in data engineering because it helps data engineers understand the condition of the data before transformation. For example, the AI can identify whether a column contains numeric values, text values, dates, or possible invalid entries. The output of this step becomes the input for the next AI model. Therefore, the first model acts as the foundation for the data quality analysis.

Step 4: Configuring the Prompt for Data Quality Issue Detection

The second AI model was used to detect data quality issues based on the profiling result from the first AI model. As shown in Figure 3.4, the prompt asks the AI model to identify null values, duplicates, format inconsistencies, outliers, and semantic value problems.

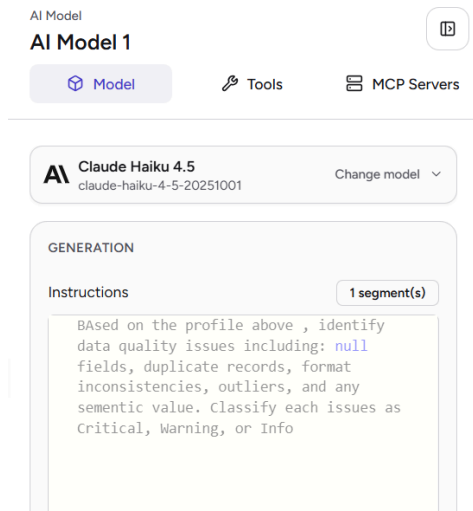


Figure 3.4: Prompt used for data quality issue detection

The prompt used for this stage was:

“Based on the profile above, identify data quality issues including null fields, duplicate records, format inconsistencies, outliers, and semantic value problems. Classify each issue as Critical, Warning, or Info.”

This step is important because it allows the AI agent to evaluate the dataset more intelligently. Traditional rule-based validation requires developers to manually write rules for each issue. However, the AI agent can detect different types of issues using the dataset context.

For example, if an age value is negative or too high, the AI can classify it as an invalid semantic value. If an email does not contain the correct format, the AI can classify it as a format issue. The use of severity levels such as Critical, Warning, and Info also helps users understand which issues need urgent attention.

Step 5: Configuring the Prompt for Report Generation

The third AI model was used to generate the final data quality report. As shown in Figure 3.5, this prompt asks the AI model to summarize the total records, issue breakdown, top issues, examples, and recommended solutions.

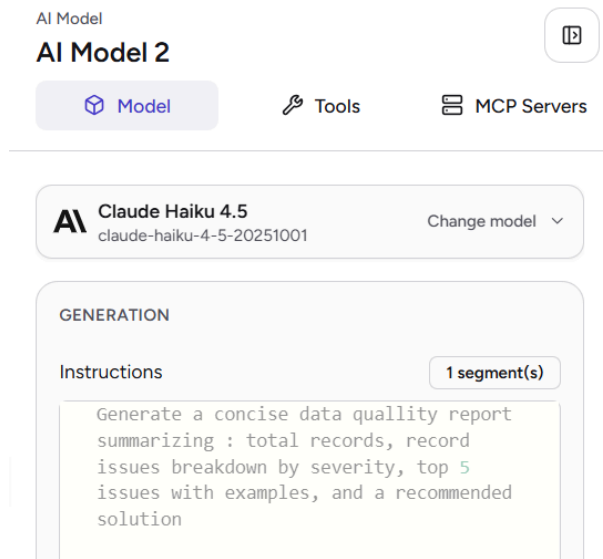


Figure 3.5: Prompt used for final data quality report generation

The prompt used for this stage was:

“Generate a concise data quality report summarizing total records, issue breakdown by severity, top issues with examples, and recommended solutions.”

This final report is useful because it converts the technical analysis into a readable summary. Instead of manually checking the raw output, users can directly review the AI-generated report to understand the dataset quality. The report helps identify what problems exist in the dataset and what actions should be taken before the data is used for further processing.

Step 6: Building the Airflow DAG

After configuring the AI agent, Apache Airflow was used to build the coding part of the pipeline. Airflow was used because it is a common orchestration tool in data engineering for managing task execution and pipeline dependencies.

As shown in Figure 3.6, the Airflow DAG contains four main tasks:



Figure 3.6: Airflow DAG workflow

The task flow shows that the data is first extracted, then loaded into a staging area, then analyzed by the Airia AI Agent, and finally cleaned using pandas. This structure is similar to a real ETL pipeline.

The DAG dependency is defined in the code as shown in Figure 3.7.

```
with DAG(
    dag_id="airia_dq_pipeline",
    description="Demo AI-assisted data quality pipeline using Airia AI Agent API",
    default_args=default_args,
    start_date=datetime(2026, 1, 1),
    schedule=None,
    catchup=False,
    tags=["demo", "airia", "data-quality"],
) as dag:
    extract_task = PythonOperator(
        task_id="extract_data",
        python_callable=extract_data,
    )

    load_task = PythonOperator(
        task_id="load_data",
        python_callable=load_data,
    )

    dq_check_task = PythonOperator(
        task_id="airia_data_quality_check",
        python_callable=airia_data_quality_check,
    )

    transform_task = PythonOperator(
        task_id="transform_data",
        python_callable=transform_data,
    )

    extract_task >> load_task >> dq_check_task >> transform_task
```

Figure 3.7: Airflow task dependency code

extract_task >> load_task >> dq_check_task >> transform_task

This code ensures that each task runs in the correct order. The transform_data task will only run after the Airia data quality check task is completed.

Step 7: Extracting and Loading the Dataset

The first task in the DAG is extract_data. This task creates or reads the raw customer dataset. The second task is load_data, which copies the dataset into a staging folder. As shown in Figure 3.8, these two tasks simulate the extraction and loading stages of a data engineering pipeline.

```

def extract_data() -> str:
    """
    Read the dummy customer CSV file.

    In this demo, the CSV is already included in the project. If the file is
    missing, the task fails so we know the input dataset was not copied into
    the Airflow DAG folder.
    """
    if not SOURCE_CSV.exists():
        raise FileNotFoundError(f"Input CSV file not found: {SOURCE_CSV}")

    print(f"Extracted CSV file: {SOURCE_CSV}")
    return str(SOURCE_CSV)

def load_data() -> str:
    """
    Simulate loading the extracted CSV file into a staging area.

    A real pipeline might load data into a database or cloud storage.
    For this demo, we copy the file into a local staging folder.
    """
    STAGING_DIR.mkdir(exist_ok=True)
    shutil.copyfile(SOURCE_CSV, STAGED_CSV)

    print(f"Loaded CSV into staging: {STAGED_CSV}")
    return str(STAGED_CSV)

```

Figure 3.8: Code for extract_data and load_data tasks

The staging folder represents a temporary area where raw data is stored before quality checking and transformation. In a real data engineering environment, this staging area could be a database, data lake, data warehouse, or cloud storage. This step is important because it separates raw data from cleaned data. It also makes the pipeline easier to manage and debug.

Step 8: Calling the Airia AI Agent API

The main AI-assisted step is handled by the `airia_data_quality_check` task. In this task, the staged CSV file is read as text and sent to the Airia AI Agent through an API call.

As shown in Figure 3.9, the code uses the Python requests library to send the CSV content to Airia.

```
def airia_data_quality_check() -> None:
    """
    Send the CSV content to Airia for an AI-assisted data quality check.

    Important: the CSV content is sent directly in userInput because the Airia
    API cannot read a local file path from your computer.
    """
    api_key = os.getenv("AIRIA_API_KEY")
    if not api_key:
        raise Exception("AIRIA_API_KEY environment variable is not set.")

    csv_content = STAGED_CSV.read_text(encoding="utf-8")

    user_input = f"""
Please perform a data quality check on this customer CSV data.

Look for missing values, invalid emails, invalid ages, invalid dates,
duplicate records, missing countries, and negative spending amounts.

Return a short report. Clearly mention whether there are critical issues.
```

```
CSV content:
{csv_content}
"""
    payload = {
        "userInput": user_input,
        "asyncOutput": False,
    }

    headers = {
        "X-API-KEY": api_key,
        "Content-Type": "application/json",
    }

    response = requests.post(
        AIRIA_ENDPOINT,
        headers=headers,
        json=payload,
        timeout=60,
    )
    response.raise_for_status()

    try:
        response_data = response.json()
    except ValueError:
        response_data = {"raw_response": response.text}

    print("Airia data quality report:")
    print(json.dumps(response_data, indent=2))

    if _response_contains_critical_issues(response_data):
        print(
            "Critical data quality issues detected by Airia. "
            "Continuing to transform_data so the demo can clean the dataset."
        )
        return
    print("No critical data quality issues detected. Continuing pipeline.")
```

Figure 3.9: Python code for calling the Airia AI Agent API

The CSV content is sent through the userInput field. This is because the Airia API cannot directly access a local file path from the user's computer. Therefore, the Python code reads the local CSV file and passes the actual dataset content to the AI agent. The API key is not hardcoded in the Python file. Instead, it is stored as an environment variable named AIRIA_API_KEY. This is a safer approach because the API key is not exposed in the source code.

Step 9: Receiving the AI-Generated Data Quality Report

After the API request is sent, Airia returns an AI-generated data quality report. The report contains the issues detected by Claude Haiku 4.5 inside the Airia agent. As shown in Figure 3.10, the Airflow log displays the response returned by the Airia AI Agent.

A screenshot of an Airflow task log for a task named 'Pre Execute'. The log shows a series of INFO messages from the Airia agent. The messages include a timestamp [2026-06-13 00:59:54] and a log level INFO. The log content is as follows:

```
13 [2026-06-13 00:59:54] INFO - Airia data quality report:
14 [2026-06-13 00:59:54] INFO - {
15 [2026-06-13 00:59:54] INFO -   "$type": "string",
16 [2026-06-13 00:59:54] INFO -   "result": "# Data Quality Report - Customer Dataset\n\n**Report Generated:** 2026-06-12 \n**Total Records Analyzed:** 60\n\n---\n\nr
17 [2026-06-13 00:59:54] INFO -   "isGuardrailBlock": false,
18 [2026-06-13 00:59:54] INFO -   "report": null,
19 [2026-06-13 00:59:54] INFO -   "isBackupPipeline": false,
20 [2026-06-13 00:59:54] INFO -   "executionId": "58029842-143c-4953-8401-91367c05e3da",
21 [2026-06-13 00:59:54] INFO -   "userId": "4785c0d3-3c5b-4ec8-bead-ac4881c4bde0",
22 [2026-06-13 00:59:54] INFO -   "files": [],
23 [2026-06-13 00:59:54] INFO -   "images": []
24 [2026-06-13 00:59:54] INFO - }
25 [2026-06-13 00:59:54] INFO - Critical data quality issues detected by Airia. Continuing to transform_data so the demo can clean the dataset.
26 [2026-06-13 00:59:54] INFO - Done. Returned value was: None
```

The log is displayed in a light blue background with a scroll bar on the right side.

Figure 3.10: Airflow log showing the AI-generated data quality report

The report includes detected issues such as missing values, invalid email formats, duplicate records, unrealistic age values, invalid dates, and negative spending amounts. This proves that the AI agent successfully analyzed the dataset and generated useful data quality insights.

In this pipeline design, the AI agent is used to detect and report issues, but it does not stop the pipeline. This design was chosen because the actual data cleaning is performed in the next step using Python and pandas.

Step 10: Cleaning the Dataset Using pandas

After the AI agent reports the data quality issues, the transform_data task performs the actual data cleaning using pandas. As shown in Figure 3.11, pandas is used to apply consistent rule-based cleaning operations.

```
def transform_data() -> str:
    """
    Clean and transform the staged data after the AI data quality report.

    For this demo, the AI agent reports the issues and this Python task applies
    simple, explainable cleaning rules with pandas.
    """
    OUTPUT_DIR.mkdir(exist_ok=True)

    df = pd.read_csv(STAGED_CSV)
    original_count = len(df)

    # Simple example transformations for tutorial purposes.
    df["name"] = df["name"].str.strip()
    df["email"] = df["email"].str.lower().str.strip()
    df["country"] = df["country"].fillna("Unknown")
    df["age"] = pd.to_numeric(df["age"], errors="coerce")
    df["total_spend"] = pd.to_numeric(df["total_spend"], errors="coerce")
    df["signup_date"] = pd.to_datetime(df["signup_date"], errors="coerce")

    # Remove duplicate customer records.
    df = df.drop_duplicates()

    # Keep rows that pass the most important quality rules.
    valid_email = df["email"].str.contains(r"^[^@\\s]+@[^@\\s]+\\.([^@\\s]+)$", na=False)
    valid_age = df["age"].between(0, 120)
    valid_date = df["signup_date"].notna()
    valid_spend = df["total_spend"].ge(0)
    df = df[valid_email & valid_age & valid_date & valid_spend].copy()

    # Fill remaining non-critical missing values.
    median_age = df["age"].median()
    if pd.notna(median_age):
        df["age"] = df["age"].fillna(median_age)
    df["country"] = df["country"].replace("", "Unknown").fillna("Unknown")

    # Save dates in a simple CSV-friendly format.
    df["signup_date"] = df["signup_date"].dt.strftime("%Y-%m-%d")

    df.to_csv(TRANSFORMED_CSV, index=False)
```

Figure 3.11: pandas data cleaning code

This design is more practical because AI is used for flexible issue detection and reporting, while pandas is used for reliable and repeatable data cleaning. The AI agent helps identify what is wrong with the dataset, but pandas performs the actual cleaning based on clear rules. This separation of responsibility makes the pipeline easier to explain and more suitable for a data engineering workflow.

Step 11: Saving the Cleaned Output

After the cleaning process is completed, the cleaned dataset is saved as `customer_data_clean.csv` in the output folder. As shown in Figure 3.12, the Airflow log confirms that the transformation process was completed successfully. The log shows that the original dataset contained 60 records, while the cleaned dataset contains 51 records after invalid and duplicate records were removed. The log also displays the location where the cleaned output file was saved.

```

► Log message source details
► Pre Execute
13 [2026-06-13 00:59:55] INFO - Original rows: 60
14 [2026-06-13 00:59:55] INFO - Clean rows written: 51
15 [2026-06-13 00:59:55] INFO - Transformed CSV written to: /home/yeewe/airflow/dags/output/customer_data_clean.csv
16 [2026-06-13 00:59:55] INFO - Done. Returned value was: /home/yeewe/airflow/dags/output/customer_data_clean.csv
► Post Execute

```

Figure 3.12: Airflow log showing successful generation of the cleaned output file

The output file was then inspected to verify the cleaning results. As shown in Figure 3.13, the cleaned dataset contains only valid records after the data cleaning process was performed using pandas. Invalid email formats, duplicate records, unrealistic age values, invalid dates, and negative spending values were removed from the dataset.

```

customer_data_clean.csv
1  customer_id,name,email,age,signup_date,country,total_spend
2  1,Aisha Rahman,aisha.rahman@example.com,22.0,2025-01-15,Malaysia,120.5
3  6,Farid Khan,farid.khan@example.com,41.0,2025-03-18,Unknown,0.0
4  7,Aisha Rahman,aisha.rahman@example.com,22.0,2025-01-15,Malaysia,120.5
5  9,Hazim Noor,hazim.noor@example.com,27.0,2025-04-12,Malaysia,77.2
6  10,Irene Koh,irene.koh@example.com,31.0,2025-04-15,Singapore,188.0
7  11,Jason Yap,jason.yap@example.com,45.0,2025-04-18,Malaysia,402.35
8  12,Kavitha Nair,kavitha.nair@example.com,38.0,2025-04-20,India,215.8
9  13,Lucas Chen,lucas.chen@example.com,24.0,2025-04-25,Malaysia,95.6
10 14,Maya Abdullah,maya.abdullah@example.com,33.0,2025-04-27,Malaysia,166.4
11 15,Nora Ismail,nora.ismail@example.com,28.0,2025-05-01,Malaysia,140.0
12 16,Omar Aziz,omar.aziz@example.com,52.0,2025-05-04,Brunei,520.1
13 17,Prisha Kumar,prisha.kumar@example.com,36.0,2025-05-06,India,275.9
14 18,Qistina Zulkifli,qistina.zulkifli@example.com,19.0,2025-05-09,Malaysia,33.4
15 19,Raj Singh,raj.singh@example.com,44.0,2025-05-12,India,390.0
16 20,Siti Aminah,siti.aminah@example.com,26.0,2025-05-14,Malaysia,180.75
17 21,Thomas Lee,thomas.lee@example.com,30.0,2025-05-17,Singapore,225.3
18 22,Usha Devi,usha.devi@example.com,47.0,2025-05-19,India,315.2
19 23,Vivian Ho,vivian.ho@example.com,35.0,2025-05-22,Malaysia,260.0
20 24,Wei Ming,wei.ming@example.com,23.0,2025-05-25,Malaysia,72.9
21 25,Xavier Lim,xavier.lin@example.com,40.0,2025-05-27,Singapore,430.45

```

Figure 3.13: Sample cleaned output dataset customer_data_clean.csv

The cleaned dataset can be used for further analysis, reporting, or dashboard development. This demonstrates that the pipeline not only identifies data quality issues through the AI agent but also produces a cleaned and usable dataset through rule-based data transformation.

Step 12: Executing the Pipeline in Airflow

Finally, the Airflow DAG was executed through the Airflow user interface. As shown in Figure 3.14, all tasks completed successfully in the latest pipeline design.

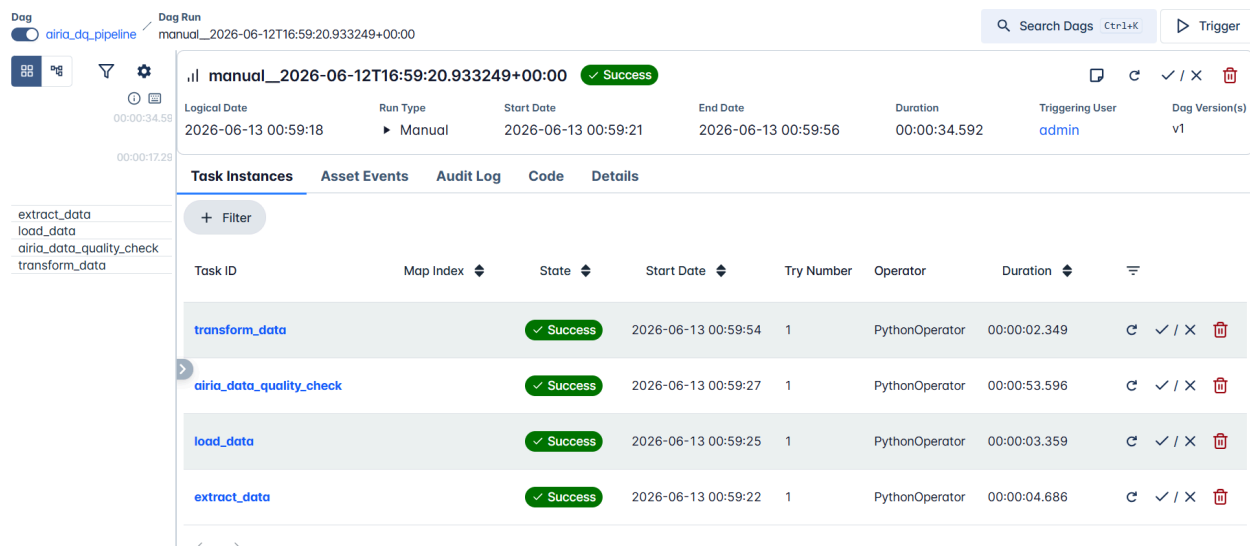


Figure 3.14: Airflow DAG execution result showing all tasks successful

The successful execution shows that the pipeline was able to complete the full process:

Extract raw data → Load to staging → AI quality report → pandas cleaning → Save cleaned output

The latest design is different from the earlier version because the Airia AI Agent no longer stops the pipeline when issues are detected. Instead, it reports the issues, and the pandas transformation step cleans the data afterwards. This approach demonstrates a complete AI-assisted data engineering pipeline where AI supports decision-making and pandas performs deterministic cleaning.

4.0 Result and Discussion

This section presents the results and discussion on overall systems including the use case design, pipeline workflow and the outcomes.

4.1 Use Case Diagram

To clearly define the functional requirements and interactions within the automated data pipeline, a system use case model was developed. Figure 4.1 shows the relationship between the Data Engineer, Airia Agent and the core automated tasks executed within the pipeline.

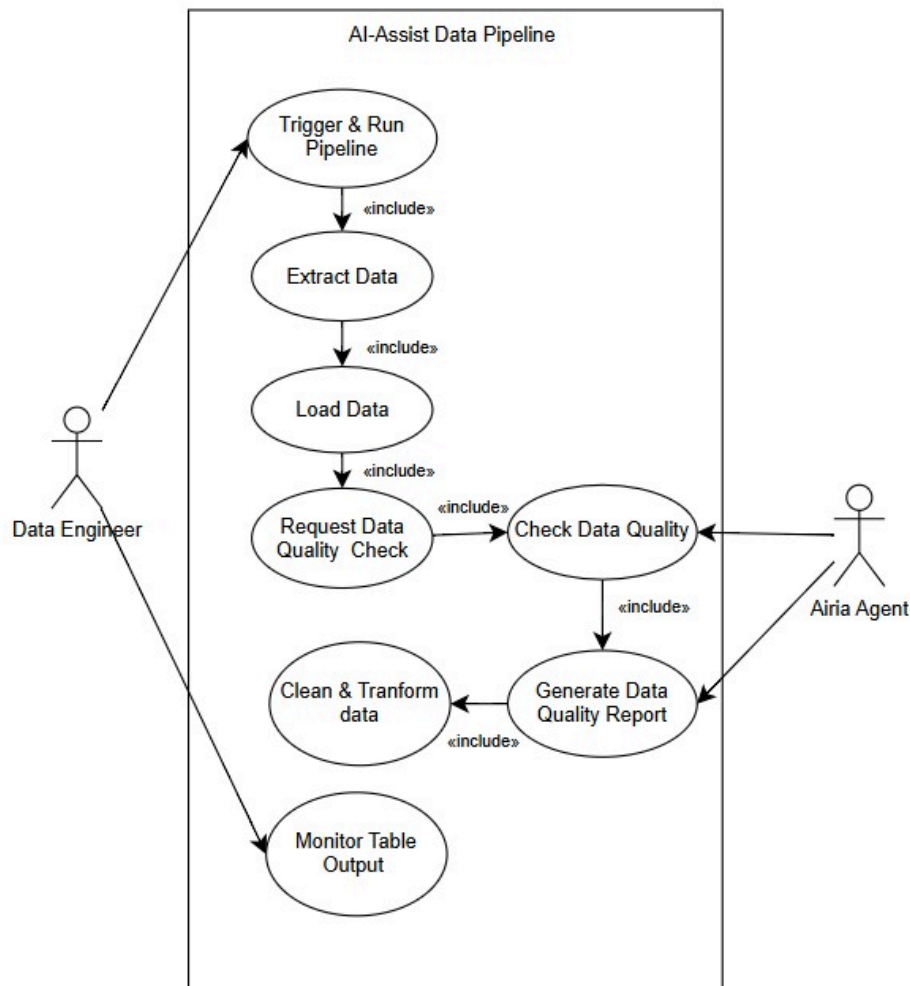


Figure 4.1 Use case diagram of AI-Assist Data Pipeline

The Data Engineer acts as the primary actor who interacts with the system to trigger, monitor, and maintain the automated workflow. The Airia Agent functions as an external system actor that handles the specialized data quality checking. To provide a clearer understanding of each use case, Table 1 maps out the specific function and description of each task within the workflow.

Use Case Name	Function
Trigger & Run Pipeline	Starts the whole automated process inside Apache Airflow.
Extract Data	Finds and reads the raw, messy customer file.
Load Data	Copies and saves the raw data into a temporary staging folder.
Request Data Quality Check	Prepares the data and sends it over to the external cloud platform.
Check Data Quality	Uses the Airia AI model to scan the data for bugs like wrong ages or bad dates.
Generate Data Quality Report	Writes down all the AI's data findings into a simple summary log.
Clean & Transform Data	Runs a Python script that automatically drops broken rows and fixes missing values.
Monitor Table Output	Check that all tasks finished successfully and see the final clean file.

Table 1: Use Case Description

4.2 Pipeline Execution Flow

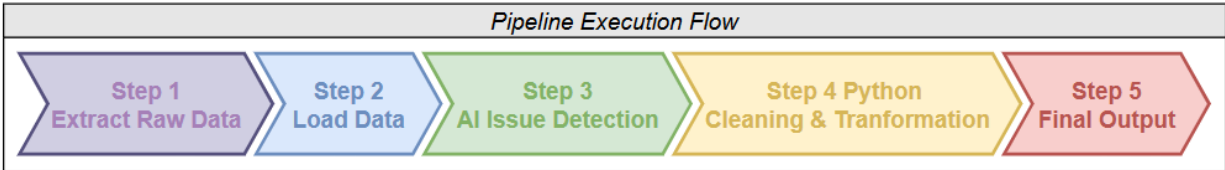


Figure 4.2 Pipeline execution flow diagram

As shown in Figure 4.2, the automated AI-assist data pipeline stages can be organized in 5 stages. The process starts with Step 1: Extract Raw Data, which is where the system locates and reads the unverified source dataset (customer_data.csv). This is immediately followed by Step 2: Load Data. The system transfers the extracted file rows to a specific local staging directory. Separating the extraction and loading activities ensures that the raw data is securely separated in a staging workspace before any cloud connectivity or alterations take place. Once the data has been prepared, the pipeline proceeds to Step 3: AI Issue Detection. This stage handles the intellectual assessment of data quality by sending the text data payload via API to the external Airia Agent by applying Claude Haiku 4.5 model, which scans for anomalies, missing values, and duplicate customer records before saving its detailed data quality findings in the system execution log. Following the automated report production, the pipeline performs Step 4: Python Cleaning & Transformation. Instead of requiring human effort from the data engineer, a pre-written Python script uses pandas' rules to remove dirty rows, clarify formatting errors, and fix small missing parameters. The cycle ends with Step 5: Final Output, which saves an analytics-ready dataset as

customer_data_clean.csv. The Airflow DAG executed successfully with all tasks complying without errors.

4.3 Result

The AI agent successfully analyzed the customer dataset and identified several data quality problems which reduced the need for manual inspection. As shown in Figure 4.3, the generated report provided clear explanations and examples of problematic records, allowing users to quickly understand the quality status of the dataset.

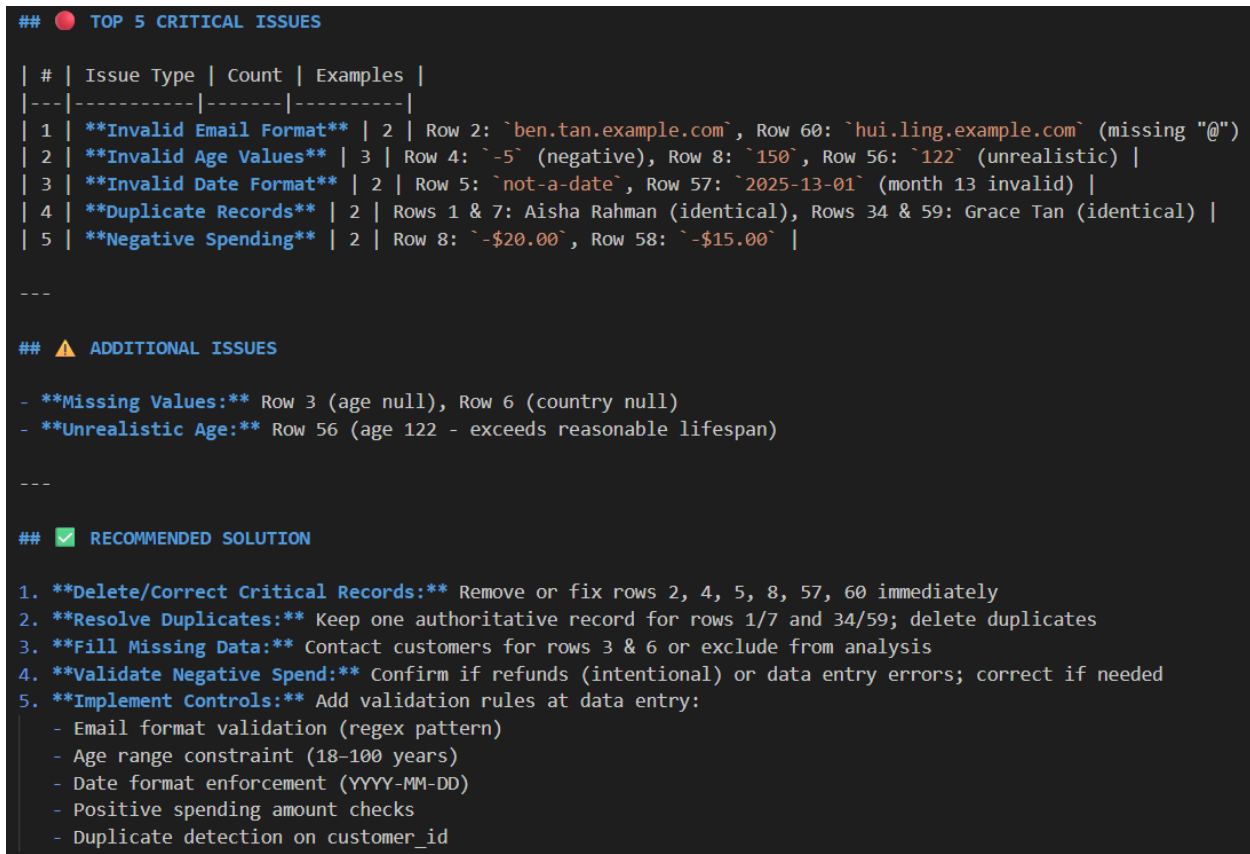


Figure 4.3 Data Quality Issues Report

In the first section, the AI model automatically groups and ranks the top data problems by priority. It provides the exact row numbers and specific bad string values, such as missing @ symbols in emails and impossible month numbers in dates. In the second section, the agent highlights minor warning errors, including missing values in Row 3 and Row 6 where fields are null. Finally, the most important part is the Recommended Solution block. The AI agent acts as an intelligent assistant by listing clear, prioritized actions for the next pipeline step. It explicitly advises removing or fixing the broken critical rows, resolving the duplicate pairs, and setting up strict programmatic data entry checks like email regex and positive spend limits. This clear, actionable layout proves that the AI agent successfully translates messy text data into useful engineering guidelines without breaking the automated pipeline run.

After Issues were identified, the pandas transformation task performed rule-based data cleaning. Table 2 shows the changes of records throughout the pipeline.

Metric Records	Value
Original Records	60
Dropped Records	9
Final Cleaned Records	51

Table 2: Records Changes During Pipeline

The reduction of records shows that the pipeline successfully filtered out the data that could negatively affect the future analysis and reporting.

This combination of AI-based issue detection and rule-based data cleaning created a practical and efficient workflow. Through this setup, this pipeline successfully achieved full automation of data quality management by running a five-stage system completely without human intervention. It also successfully eliminated manual human inspection effort, as the AI agent instantly flagged all 11 problematic rows and sorted them by priority, saving an engineer from searching through text files. Finally, the system successfully demonstrated a reliable hybrid framework by letting the AI work as a smart inspector while the pandas script acted as a strict cleanup engine. Safely dropping 9 corrupted rows to deliver 51 clean records proves that this approach creates a highly dependable, production-ready ETL workflow.

5.0 Reflection

Lau Yee Wen

This tutorial provided valuable experience in AI-assisted data engineering and helped improve understanding of how AI can be integrated into a data pipeline. Through this tutorial, knowledge was gained on using Apache Airflow and Directed Acyclic Graphs (DAGs) to manage and automate workflow execution. The pipeline was designed to perform data extraction, data loading, AI-assisted data quality checking, and data transformation in a structured sequence.

One of the main learning outcomes was understanding how an AI agent can support data quality assessment. The Airia AI Agent, powered by Claude Haiku 4.5, was able to analyze the dataset and identify issues such as missing values, duplicate records, invalid formats, unrealistic values, and negative spending amounts. This demonstrated how AI can assist data engineers in detecting data quality problems more efficiently.

Several challenges were encountered during the implementation process. Initially, difficulties occurred when connecting to the Airia API due to authentication and configuration issues. Different approaches were tested before a successful connection was established. This provided a better understanding of API integration, request headers, payload formats, and API key management. Another valuable experience was learning how to trigger the AI agent through Python code instead of only running it within the Airia platform. This showed how AI services can be integrated into an automated workflow and executed as part of a data engineering pipeline.

In addition, practical experience was gained in setting up and running Apache Airflow using WSL Ubuntu. Although the setup process was challenging at first, it provided exposure to tools commonly used in real-world data engineering environments.

Overall, this tutorial improved understanding of AI-assisted pipelines, workflow orchestration, API integration, and data quality management. It demonstrated how AI can be used to identify data quality issues, while Python and pandas can be used to clean and transform data. The experience highlighted the potential of combining AI and traditional programming techniques to build more efficient and intelligent data engineering workflows.

Chau Ying Jia

This tutorial provides new knowledge and experience in developing an AI-assisted data engineering pipeline. A better understanding was gained on how Airia AI Agent and Claude Haiku 4.5 can be used to support data quality checking, issue detection and report generation. The AI agent was able to identify problems such as missing values, duplicate records, invalid email formats, unrealistic age values, invalid dates and negative spending amounts after prompting them in normal texts. It is so convenient and easy to use for people who do not have a data engineering background but need to make a data quality check on their data.

Besides that, practical knowledge was gained in using Apache Airflow to manage the pipeline workflow. Before this, knowledge of these data orchestrations are only learned theoretically. Fortunately, we have the chance to explore using Apache Airflow. The Airflow DAG helped arrange the tasks in the correct order, from data extraction and loading to AI quality checking, data cleaning and saving the cleaned output. At first, the pipeline design was confusing because the pipeline would stop when data quality issues were detected in the raw data. However, after discussing among teammates and re-prompting the AI agent, it was later understood by the AI agent it should be used to report the issues and not detect the issue then stopped. After that, it will pass to the python for data cleaning and data transformation.

Several challenges were faced during the implementation, especially when connecting Python code with the Airia API. Some configuration issues and mismatches occurred but they were successfully solved. Through this process, knowledge was gained about API keys, request headers and API responses. The tutorial also showed that AI and rule-based programming can work together effectively. AI was useful for detecting and explaining data problems, while using Python was more suitable for applying consistent cleaning and transformation rules.

Overall, this tutorial was meaningful because it provided hands-on experience with AI-assisted pipelines, Airflow orchestration, API integration and Python data cleaning. Although the process was challenging at first as it did not have a direct tutorial teaching step by step working. However, it really helped improve technical understanding and problem-solving skills. It also showed that AI can support data engineering workflows but proper programming logic is still needed to produce consistent, clean and reliable data.

Poh Lok Yee

This tutorial offered a practical introduction to AI-assisted data engineering that showing how artificial intelligence can fit into automated data pipelines. This project focuses on setting up a workflow where a messy customer dataset was ingested, scanned for quality issues by an AI agent, and then cleaned using Python scripts before being saved. The instructional videos provided a helpful overview of how the Airia platform works and connects with other tools and a whole setup in action really teaches how data engineering is shifting from manually written validation rules to modern pipelines that use intelligent language models to handle initial data inspections.

Building this pipeline provided great hands-on experience with data orchestration and combining AI with traditional programming. Using Apache Airflow and Directed Acyclic Graphs (DAGs) made it easy to see how tasks should be sequenced and executed in the correct order. It was especially interesting to see how the AI agent, powered by Claude Haiku 4.5 could automatically profile the dataset and catch complex flaws like duplicate records, formatting errors, unrealistic ages, and negative numbers using plain text prompts. This showed how convenient AI tools are for speeding up data quality checks without needing extensive code right from the start.

However, getting everything to connect properly was quite challenging at first due to a lack of familiarity with the platform integrations. Moving from just understanding the concept to actually linking a local Python environment with external cloud services threw up some tricky configuration problems. Figuring out how to pass request headers and payloads through an Airflow operator to the Airia API was confusing in the beginning, leading to a few connection mismatches. On top of that, the initial pipeline logic caused some confusion because the workflow was set to stop completely the moment the AI found an error. This required a shift in thinking to realize the AI should only handle the reporting, leaving the actual cleaning to Python scripts.

In short, the tutorial proved that while AI is incredibly helpful for spotting and explaining data flaws, solid programming logic is still essential for producing clean, reliable datasets. Letting the AI handle the flexible reporting while using pandas for the strict cleaning rules allowed the pipeline to finish in a successful 34.59 seconds without any crashes. Successfully bringing the dataset down from 60 messy rows to 51 clean records proved that smart tools and traditional code work best together. This project gave a great entry-level look into workflow orchestration, API integration, and real-world problem-solving in data engineering which delivering a strong foundation for building and troubleshooting modern data engineering pipelines in the future.

6.0 References

Srikanth Peddisetti. (2023). *AI-Driven Data Engineering: Streamlining Data Pipelines for Seamless Automation in Modern Analytics*. 15(01), 1066–1075. <https://doi.org/10.70153/ijcmi/2023.15101>

Yaman Tandon. (2025). AI-powered data engineering: How machine learning is revolutionizing ETL and data pipelines. *World Journal of Advanced Engineering Technology and Sciences*, 15(3), 118–125. <https://doi.org/10.30574/wjaets.2025.15.3.0858>